

India Weather & Air Quality ETL Pipeline

Technical Design Specification — Airflow · MongoDB · PostgreSQL

September 2026 · v1.0

Contents

1 Purpose of This Document	3
2 Project Goals	3
3 Architecture Overview	3
4 Data Sources	4
4.1 Open-Meteo Forecast API — Weather	4
4.2 WAQI / aqicn.org — Air Quality	5
4.3 Indian City Master Data — Messy CSV	6
5 Data Model	7
5.1 MongoDB — Raw / Landing Zone	7
5.2 PostgreSQL — Curated Warehouse (star schema)	7
6 Transformation & Data Quality Logic	9
6.1 City name reconciliation	9
6.2 Unit & type standardization	9
6.3 Deduplication	9
6.4 Data quality check catalog	10
7 Airflow Orchestration Design	10
7.1 DAG 1 — dim_city_pipeline (slow-changing dimension)	10
7.2 DAG 2 — weather_aqi_pipeline (main pipeline)	11
7.3 Retry, backoff, and partial-failure handling	12
7.4 Idempotency	12
7.5 Observability	12
8 Technology Stack	13
9 Repository Structure	13
10 Environment & Docker Compose Setup	14
11 Testing Plan	16
12 Build Plan (recommended order, with acceptance criteria)	16
13 Stretch Goals (post-v1, optional)	17

14 Resume & Interview Talking Points	17
15 Example Analytical Queries (for SQL practice on your own data)	18
16 Data Usage & Attribution	19

1 Purpose of This Document

This is a build specification for an end-to-end data engineering portfolio project: a multi-source, multi-city ETL pipeline orchestrated by Apache Airflow, built to demonstrate entry-level-to-junior data engineering skills for a resume/interview context (India, 2026 hiring cycle).

It is written to be handed directly to a coding agent (e.g. Antigravity) or followed manually. It specifies **what** to build and **why**, with enough precision that implementation choices are unambiguous. Where exact tool versions may have moved on by build time, that is called out explicitly so the agent checks current versions rather than assuming.

Read this before writing any code. Section 12 (Build Plan) gives the recommended build order with acceptance criteria per milestone — build in that order, verifying each milestone before moving to the next.

2 Project Goals

Primary goals (what makes this “more than a basic ETL demo”):

1. **Genuine multi-source ingestion** — two live third-party APIs with independently-drifting schemas, plus a static but deliberately messy CSV, reconciled into one clean model.
2. **Real transformation logic** — fuzzy entity resolution (city name reconciliation), unit standardization, deduplication, not just pass-through copying.
3. **A real data-quality gate** — automated checks that catch bad data and quarantine it, rather than silently loading it.
4. **Idempotent, incremental loads** — rerunning the pipeline for a period it already processed must not duplicate data.
5. **Resilience** — retries with exponential backoff on flaky network calls, and graceful partial-failure handling (one dead API endpoint should not fail the whole run).
6. **Observability** — the pipeline logs its own run history (rows processed, rows rejected, failures) to a queryable table.
7. **Fully free to run** — no AWS/Azure account, no cloud billing account required. Everything runs locally via Docker Compose.

Explicit non-goals for v1 (call these out as “future work” in the README — don’t build them now):

- Streaming/Kafka ingestion (batch-only for v1; mentioned only as a possible stretch goal).
- Kubernetes deployment.
- ML forecasting on the collected data.
- A cloud data warehouse load (BigQuery) — listed as an optional stretch goal in Section 13, not part of the core build.

3 Architecture Overview

The pipeline uses a **two-zone architecture**: a raw/landing zone in MongoDB that stores API responses close to their original shape, and a curated warehouse in PostgreSQL that stores cleaned, validated, dimensionally-modeled data. This is a small-scale version of the “bronze/silver” pattern used in real data platforms, and it directly motivates *why* two different databases are used (not just “because the brief said so”): the two sources have genuinely different, evolving JSON shapes that don’t belong in a rigid relational schema until they’ve been cleaned.

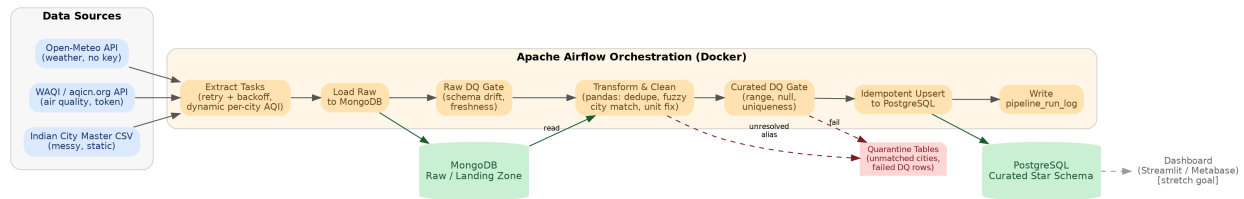


Figure 1: Architecture overview

Flow, in words:

1. Airflow tasks call the Open-Meteo weather API (one batched call for all cities) and the WAQI air-quality API (one call per city, since WAQI has no batch endpoint).
2. Responses are written to MongoDB almost as-is — this is the audit trail / raw zone. If a transformation rule turns out to be wrong later, the original data is still there to reprocess.
3. A raw data-quality gate checks the *shape* of what came back (expected keys present, response not empty, not stale) before anything downstream trusts it.
4. A transform step cleans and standardizes the data: unit conversions, null handling, and — the most interesting part — fuzzy-matching incoming city name strings against a canonical city dimension (because “Bengaluru”, “Bangalore”, and “BANGALORE” all need to resolve to the same city).
5. A curated data-quality gate checks *values* (ranges, nulls in critical fields, uniqueness) before loading. Rows that fail are quarantined, not dropped silently and not allowed to poison the warehouse.
6. Clean data is upserted into a PostgreSQL star schema, keyed so that reruns overwrite rather than duplicate.
7. Every run writes a summary row to a pipeline_run_log table — this is what lets you say in an interview “here’s how I’d know if the pipeline silently broke.”

4 Data Sources

4.1 Open-Meteo Forecast API — Weather

Base URL	https://api.open-meteo.com/v1/forecast
Auth	None required
Cost	Free for non-commercial use, no signup
Rate limit	Generous (order of thousands of calls/day for non-commercial use); irrelevant at this project’s volume
Batching	Yes — pass comma-separated latitude and longitude lists in one request; response is an array with one block per coordinate pair

Example request (all cities batched into one call):

```
GET https://api.open-meteo.com/v1/forecast
?latitude=28.6139,19.0760,12.9716
&longitude=77.2090,72.8777,77.5946
&current=temperature_2m,relative_humidity_2m,apparent_temperature,
precipitation,weather_code,wind_speed_10m
&timezone=Asia%2FKolkata
```

Example response shape (per city, current block):

```
{
  "latitude": 28.61,
  "longitude": 77.21,
  "timezone": "Asia/Kolkata",
  "current": {
    "time": "2026-09-26T14:00",
    "temperature_2m": 33.4,
    "relative_humidity_2m": 41,
    "apparent_temperature": 35.1,
    "precipitation": 0.0,
    "weather_code": 1,
    "wind_speed_10m": 11.2
  }
}
```

weather_code is a WMO numeric code (0 = clear sky, 61 = slight rain, 95 = thunderstorm, etc.) — the transform step maps this to a human-readable description; keep the mapping table in include/transform/clean_weather.py.

4.2 WAQI / aqicn.org — Air Quality

Base URL	https://api.waqi.info/feed/geo:{lat};{lon}/
Auth	Free token, query param ?token=... — sign up at aqicn.org/data-platform/token/
Cost	Free
Rate limit	1,000 requests/second (not a real constraint here)
Batching	No — one call per city, by design (this is what gives the project a genuine reason to use per-city retry logic and Airflow dynamic task mapping)

Query by geo-coordinate (not city name) so it always resolves to the nearest official monitoring station regardless of spelling:

GET https://api.waqi.info/feed/geo:28.6139;77.2090/?token=YOUR_TOKEN

Example response (abbreviated):

```
{
  "status": "ok",
  "data": {
    "aqi": 187,
    "idx": 1437,
    "city": { "name": "Anand Vihar, Delhi", "geo": [28.647, 77.316] },
    "dominentpol": "pm25",
    "iaqi": {
      "pm25": { "v": 187 }, "pm10": { "v": 142 },
      "no2": { "v": 24 }, "so2": { "v": 8 },
      "co": { "v": 6.1 }, "o3": { "v": 12 }
    },
    "time": { "s": "2026-09-26 14:00:00", "tz": "+05:30" }
  }
}
```

Note the structural difference from Open-Meteo: pollutants are nested under `iaqi.<pollutant>.v`, timestamps are formatted differently, and there is a `dominentpol` field with no Open-Meteo equivalent. **This is the real schema drift** the transform layer has to reconcile — it isn't contrived.

Why WAQI specifically, not a second global model: WAQI's Indian stations are sourced from the CPCB (Central Pollution Control Board) ground-monitoring network — the same stations behind India's official AQI. Open-Meteo's own air-quality endpoint (CAMS) is a global atmospheric model and can diverge substantially from ground truth during pollution events (well-documented for Delhi-NCR in winter). Neither the IMD (India Meteorological Department) nor CPCB expose a public developer API directly, so WAQI is the closest free, programmatic approximation of official Indian AQI. **Document this sourcing decision in the README** — it's a legitimate data-engineering judgment call and a good interview talking point.

Terms of service — respect these: data cannot be resold, used in a paid app, or redistributed as a cached/archived dataset; attribution to the World Air Quality Index project is required. This is fine for a personal portfolio project — note it explicitly in the README's data sources section.

4.3 Indian City Master Data — Messy CSV

This is the “handle imperfect data” showcase. Build it in two steps:

1. **Real base data.** Source latitude/longitude, state, and population for ~20 Indian cities spanning tier-1 and tier-2 (e.g. Mumbai, Delhi, Bengaluru, Chennai, Kolkata, Hyderabad, Pune, Ahmedabad, Jaipur, Lucknow, Surat, Kanpur, Nagpur, Indore, Patna, Bhopal, Chandigarh, Guwahati, Kochi, Coimbatore) from a public source (e.g. a Kaggle Indian-cities dataset, or data.gov.in city population tables).
2. **Deliberately introduce realistic messiness on top of it**, and say so plainly in the README (“synthetic messiness layered on real base data, to exercise the cleaning/validation logic”). This is intellectually honest and is itself a good interview point — it shows you can construct test cases for data quality logic, not just react to messiness you happened to find.

Required messy patterns to include in `data/raw/indian_cities_master_messy.csv` (the transform layer must be written to handle every one of these):

Pattern	Example
Inconsistent casing/whitespace Legacy/alternate city names	"Bengaluru", "BENGALURU", "bengaluru " Bombay → Mumbai, Calcutta → Kolkata, Madras → Chennai, Poona → Pune, Bangalore → Bengaluru, Cochin → Kochi
Mixed null tokens	"", "NA", "N/A", "null", "-", "999999" all meaning “missing”
Inconsistent state formatting Numbers as dirty strings	"Uttar Pradesh", "UP", "uttar pradesh" population as "12,442,373" (thousands separators)
Impossible placeholder values	population -1, or a lat/long pair outside India's bounding box (data-entry error)
Swapped columns	one row with latitude/longitude values transposed
Exact and near-duplicate rows	one exact duplicate row, one same-city-different-casing duplicate

Pattern	Example
Fully blank rows	1-2 empty rows (common CSV export artifact)

5 Data Model

5.1 MongoDB — Raw / Landing Zone

Three collections, one document per API call (or per CSV ingestion batch). Store the response close to as-received, plus lightweight metadata for traceability.

```
// raw_weather_readings
{
  "_id": ObjectId,
  "source": "open-meteo",
  "run_id": "scheduled__2026-09-26T12:00:00+00:00",
  "fetched_at": "2026-09-26T12:00:04Z",
  "request_params": { "latitude": "28.6139,...", "longitude": "77.2090,..." },
  "raw_response": { /* full API JSON */ }
}

// raw_aqi_readings
{
  "_id": ObjectId,
  "source": "waqi",
  "run_id": "scheduled__2026-09-26T12:00:00+00:00",
  "city_hint": "Delhi",
  "fetched_at": "2026-09-26T12:00:11Z",
  "raw_response": { /* full API JSON, or {"status":"error", ...} on failure */ }
}

// raw_city_master_batches
{
  "_id": ObjectId,
  "ingested_at": "2026-09-20T09:00:00Z",
  "source_file": "indian_cities_master_messy.csv",
  "row_count": 24,
  "raw_rows": [ { /* row as dict, untouched */ }, ... ]
}
```

Upsert raw documents on (source, city_hint or coordinate key, run_id) so retried tasks within the same Airflow run overwrite instead of duplicating. Index on run_id and fetched_at for cleanup/debugging queries.

5.2 PostgreSQL — Curated Warehouse (star schema)

Run this in a **separate PostgreSQL container from Airflow's own metadata database**. Coupling your analytical warehouse to Airflow's internal metadata DB is a real anti-pattern worth deliberately avoiding — mention this decision in the README.

Table	Type	Purpose
dim_city	Dimension	Canonical city list — one row per real city

Table	Type	Purpose
dim_city_alias	Lookup	Every known raw-name variant → canonical city_id
dim_date	Dimension	Standard date dimension for easy time-based rollups
fact_weather_reading	Fact	One row per city per timestamp, weather metrics
fact_air_quality_reading	Fact	One row per city per timestamp, AQI + pollutant breakdown
pipeline_run_log	Observability	One row per DAG run, with counts and status
dq_check_results	Observability	One row per data-quality check per run
quarantine_unmatched_city_names	Quarantine	Raw city strings that couldn't be resolved to a city_id

dim_city

Column	Type	Notes
city_id	SERIAL PK	
city_name	VARCHAR(100) UNIQUE	canonical name
state	VARCHAR(100)	
region	VARCHAR(20)	North / South / East / West / Central / Northeast
tier	SMALLINT	1, 2, or 3
population	BIGINT	nullable
latitude	NUMERIC(9,6)	
longitude	NUMERIC(9,6)	
is_active	BOOLEAN	default true
created_at / updated_at	TIMESTAMPTZ	

dim_city_alias — alias_id PK, raw_name_variant VARCHAR UNIQUE, city_id FK, match_method (manual or fuzzy), match_score NUMERIC (nullable, populated when fuzzy-matched).

dim_date — standard: date_id INT PK (YYYYMMDD), full_date DATE UNIQUE, year, month, month_name, day, day_of_week, week_of_year, is_weekend.

fact_weather_reading — id BIGSERIAL PK, city_id FK, date_id FK, reading_ts TIMESTAMPTZ, temperature_c NUMERIC(5,2), apparent_temperature_c NUMERIC(5,2), humidity_pct SMALLINT, wind_speed_kmh NUMERIC(5,2), precipitation_mm NUMERIC(6,2), weather_code SMALLINT, weather_description VARCHAR(50), source VARCHAR(20) DEFAULT 'open-meteo', load_ts TIMESTAMPTZ DEFAULT now(). **UNIQUE (city_id, reading_ts)** — this constraint is what makes loads idempotent.

fact_air_quality_reading — id BIGSERIAL PK, city_id FK, date_id FK, reading_ts TIMESTAMPTZ, aqi_value SMALLINT, aqi_category VARCHAR(30), dominant_pollutant VARCHAR(10), pm25, pm10, no2, so2, co, o3 (all NUMERIC(6,2)), station_name VARCHAR(150), source VARCHAR(20) DEFAULT 'waqi', load_ts TIMESTAMPTZ DEFAULT now(). **UNIQUE (city_id, reading_ts)**.

pipeline_run_log — run_id BIGSERIAL PK, dag_id VARCHAR, task_id VARCHAR, logical_date TIMESTAMPTZ, started_at, ended_at TIMESTAMPTZ, status VARCHAR (success/partial/failed), rows_extracted INT, rows_loaded INT, rows_rejected INT, cities_failed INT, error_message TEXT.

dq_check_results — id BIGSERIAL PK, run_id FK, check_name VARCHAR, table_name VARCHAR, status VARCHAR (pass/warn/fail), details TEXT, checked_at TIMESTAMPTZ.

quarantine_unmatched_city_names — id BIGSERIAL PK, raw_name VARCHAR, source VARCHAR, first_seen_at, last_seen_at TIMESTAMPTZ, occurrence_count INT, resolved BOOLEAN DEFAULT false.

Put the DDL for all of these in sql/001_*.sql through sql/008_*.sql, one file per table, numbered so they can be applied in order.

6 Transformation & Data Quality Logic

6.1 City name reconciliation

This is the single most interesting transform in the project — worth building carefully and explaining well.

1. Normalize the incoming string: strip whitespace, collapse internal spaces, lowercase for comparison.
2. Look up against dim_city_alias (exact match on normalized string) — this handles known variants instantly (Bombay, Calcutta, Madras, Poona, Bangalore, Cochin, and any casing variant already seen before).
3. If no exact alias match, fuzzy-match against dim_city.city_name using rapidfuzz (token-sort ratio). Accept if score ≥ 90 ; write the match back into dim_city_alias with match_method='fuzzy' so it's an exact match next time (the alias table *learns*).
4. If still no match above threshold, write the raw string into quarantine_unmatched_city_names (upsert incrementing occurrence_count), and drop that reading for this run rather than guessing wrong. This row is exactly what you'd review in a "why is this city missing from my dashboard" investigation — a good interview story.

6.2 Unit & type standardization

- Weather: Open-Meteo already returns metric units (°C, km/h, mm) — no conversion needed, but validate types and coerce.
- AQI: WAQI pollutant values may arrive as strings or "-" for unavailable sensors — coerce to numeric or null explicitly, never let a "-" silently become 0.
- Timestamps: normalize both sources to UTC TIMESTAMPTZ; Open-Meteo returns local time strings, WAQI returns a timestamp with its own UTC offset — get this conversion right and unit-test it, it's an easy silent bug.

6.3 Deduplication

- Within a single run, if the same city produces more than one reading (shouldn't happen but defend against it), keep the latest by fetched_at.
- Across runs, the UNIQUE (city_id, reading_ts) constraint plus ON CONFLICT ... DO UPDATE upsert handles it at the load step.

6.4 Data quality check catalog

Implement each as a small, independently testable Python function in `include/quality/dq_checks.py`. Every check writes a row to `dq_check_results`.

Check	Applies to	Rule	On failure
schema_presence	raw API response	expected top-level keys exist	retry once, else mark city failed for this run
freshness	fact tables, post-load	$\max(\text{reading_ts})$ within schedule interval + 30 min	warn
completeness	run-level	$\geq 90\%$ of active cities have a reading	warn at 80–90%, fail below 80%
null_critical_fields	fact rows	city_id , reading_ts , ≥ 1 metric not null	drop row → quarantine
range_temperature	fact_weather	$-10\text{ }^{\circ}\text{C} \leq \text{temperature_c} \leq 55\text{ }^{\circ}\text{C}$	drop row → quarantine
range_humidity	fact_weather	$0 \leq \text{humidity_pct} \leq 100$	drop row → quarantine
range_aqi	fact_air_quality	$0 \leq \text{aqi_value} \leq 500$	drop row → quarantine
row_count_sanity	run-level	loaded rows $\approx$ expected active cities $\pm$ tolerance	warn
city_resolution	transform step	every raw city identifier resolves to a city_id	unresolved → quarantine table, row dropped
uniqueness	fact tables, post-load	no duplicate (city_id , reading_ts)	should be structurally impossible given the upsert key — check anyway as a sanity assertion

This is deliberately built as plain Python functions rather than a Great Expectations suite — simpler to write, test, and *explain line-by-line in an interview* at entry level. Section 13 lists swapping in Great Expectations as an optional stretch goal once the core pipeline is solid.

7 Airflow Orchestration Design

Two DAGs, because dimension data and fact data change at very different rates and coupling them is a common design smell worth deliberately avoiding.

7.1 DAG 1 — `dim_city_pipeline` (slow-changing dimension)

- **Schedule:** manual trigger / @weekly (cities essentially never change).
- Reads the messy CSV, cleans it, resolves aliases, upserts into `dim_city` and `dim_city_alias`, logs unresolved names to quarantine.
- Kept deliberately separate from the frequent fact-loading DAG below.

7.2 DAG 2 — weather_aqi_pipeline (main pipeline)

- **Schedule:** `schedule="0 */3 * * *"` (every 3 hours, 8 runs/day), `catchup=False`, `max_active_runs=1`.
- At this cadence: $\sim 8 \times 20 = 160$ WAQI calls/day and ~ 8 -16 Open-Meteo calls/day — trivial against both APIs' limits. Volume is not the constraint here; correctness and resilience are.

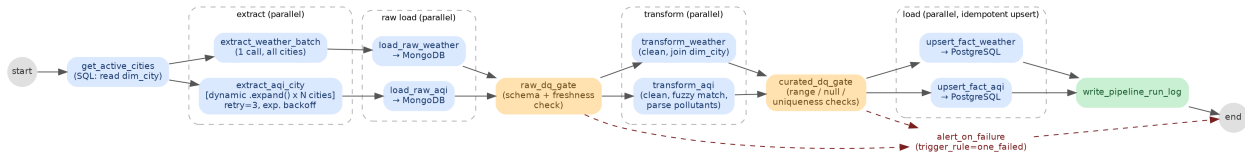


Figure 2: Main DAG task graph

Build with the **TaskFlow API** (`@dag / @task` decorators) — current idiomatic Airflow style, portable across the 2.x and 3.x lines.

```
from airflow.sdk import dag, task # Airflow 3.x import path;
                                # use airflow.decorators on 2.x
from datetime import timedelta, datetime

default_args = {
    "owner": "you",
    "retries": 3,
    "retry_delay": timedelta(minutes=2),
    "retry_exponential_backoff": True,
    "max_retry_delay": timedelta(minutes=20),
    "execution_timeout": timedelta(minutes=10),
}

@dag(
    dag_id="weather_aqi_pipeline",
    schedule="0 */3 * * *",
    start_date=datetime(2026, 9, 1),
    catchup=False,
    max_active_runs=1,
    default_args=default_args,
    tags=["etl", "weather", "aqi"],
)
def weather_aqi_pipeline():

    @task
    def get_active_cities() -> list[dict]:
        # SELECT city_id, city_name, latitude, longitude
        # FROM dim_city WHERE is_active = true
        ...

    @task
    def extract_weather_batch(cities: list[dict]) -> dict:
        # one Open-Meteo call for all cities via HttpHook,
        # provider-level retry via HttpHook's own retry args
        ...

    @task(retries=3, retry_delay=timedelta(seconds=30),
          retry_exponential_backoff=True)
    def extract_aqi_city(city: dict) -> dict:
```

```

    # one WAQI call for this city; internal try/except so a
    # single dead station returns {"status": "failed", ...}
    # instead of raising – see 7.3 for why
    ...

    cities = get_active_cities()
    weather_raw = extract_weather_batch(cities)
    aqi_raw = extract_aqi_city.expand(city=cities)    # dynamic task mapping,
                                                    # one mapped instance per city

    # ... load_raw_weather / load_raw_aqi / raw_dq_gate / transform_weather /
    #      transform_aqi / curated_dq_gate / upsert_fact_weather /
    #      upsert_fact_aqi / write_pipeline_run_log – see diagram above

weather_aqi_pipeline()

```

`extract_aqi_city.expand(city=cities)` is **dynamic task mapping** — Airflow creates one mapped task instance per city at runtime, each with its own independent retry/backoff and its own success/failure state visible in the UI. This is worth calling out explicitly in your README and in interviews: it’s a genuinely more advanced pattern than a for loop inside one task, and it’s exactly the shape real per-entity API polling takes in production.

7.3 Retry, backoff, and partial-failure handling

Two layers of resilience, and they solve different problems:

1. **Airflow-level retries** (`retries`, `retry_delay`, `retry_exponential_backoff` in `default_args/@task`) — handle transient failures (a timeout, a 500 response) by re-running the whole task after a backing-off delay.
2. **In-code partial-failure handling inside `extract_aqi_city`** — even after Airflow’s retries are exhausted for one city, that should not fail the entire mapped task set or the DAG run. Catch the exception, log it, and **return a structured failure dict** (`{"city_id": ..., "status": "failed", "error": "..."}`) instead of raising. Downstream tasks filter these out before loading, and `pipeline_run_log.cities_failed` records the count. One dead AQI station on a given run should degrade the pipeline (fewer rows loaded, logged clearly), never crash it.

This distinction — “retry the transient stuff, but never let one bad city take down the whole run” — is precisely the kind of thing the roadmap notes interviewers probe for (“what broke and how did you handle it”), so make sure you can walk through this code path fluently.

7.4 Idempotency

- **PostgreSQL:** `INSERT ... ON CONFLICT (city_id, reading_ts) DO UPDATE SET ...` for both fact tables. Re-running a logical_date you’ve already loaded overwrites those rows instead of duplicating them.
- **MongoDB:** raw documents upserted on a compound key including the Airflow `run_id`, so retries within the same run don’t create duplicate raw documents either.

7.5 Observability

Every run of `weather_aqi_pipeline` writes exactly one row to `pipeline_run_log` (rows extracted / loaded / rejected, cities failed, status, error message if any) and one row per check to `dq_check_results`. This is what turns “the pipeline ran” into “I can query whether the pipeline is healthy” — a genuinely production-flavored habit, and directly answers the kind of “how would you know if this silently broke” interview question.

Stretch: wire `on_failure_callback` to post to Slack or send an email when a task exhausts its retries — described in Section 13, not required for v1.

8 Technology Stack

Component	Choice	Notes
Orchestration	Apache Airflow, latest stable 3.x line	Check the official docs site for the exact current patch before pinning a version (URL in Section 10) — the 3.x line renamed <code>airflow-webserver</code> → <code>airflow-api-server</code> and <code>execution_date</code> → <code>logical_date</code> versus 2.x, so don't copy older 2.x tutorials verbatim
Executor	LocalExecutor	Simplest correct choice for a single-machine portfolio project — no Celery/Redis needed
Language	Python 3.12	
Raw store	MongoDB 7.x (official image)	
Warehouse	PostgreSQL 16 (official image), separate container from Airflow's metadata DB	
Transform	pandas, rapidfuzz	fuzzy city matching
HTTP + resilience	requests, tenacity (or Airflow's <code>HttpHook</code> retry args)	
Airflow providers	<code>apache-airflow-providers-postgres</code> , <code>apache-airflow-providers-mongo</code> , <code>apache-airflow-providers-http</code>	use hooks, don't hand-roll connections
Testing	pytest, responses (mock HTTP)	
Containerization	Docker + Docker Compose	
Admin UIs (dev only)	Adminer (Postgres), <code>mongo-express</code> (Mongo)	
Dashboard (stretch)	Streamlit	Section 13
CI (stretch)	GitHub Actions	Section 13

9 Repository Structure

```

weather-aqi-etl/
├── dags/
│   ├── weather_aqi_pipeline.py
│   └── dim_city_pipeline.py
└── include/

```

```

├── extract/
│   ├── open_meteo.py
│   └── waqi.py
├── transform/
│   ├── clean_weather.py
│   ├── clean_aqi.py
│   └── city_matcher.py
├── load/
│   ├── mongo_loader.py
│   └── postgres_loader.py
├── quality/
│   └── dq_checks.py
├── utils/
│   ├── logging_config.py
│   └── config.py
├── data/raw/
│   └── indian_cities_master_messy.csv
├── sql/
│   ├── 001_create_dim_city.sql
│   ├── 002_create_dim_city_alias.sql
│   ├── 003_create_dim_date.sql
│   ├── 004_create_fact_weather.sql
│   ├── 005_create_fact_aqi.sql
│   ├── 006_create_pipeline_run_log.sql
│   ├── 007_create_dq_check_results.sql
│   └── 008_create_quarantine_tables.sql
├── tests/
│   ├── test_city_matcher.py
│   ├── test_clean_weather.py
│   ├── test_clean_aqi.py
│   ├── test_dq_checks.py
│   └── test_dag_integrity.py
├── docker/
│   ├── Dockerfile.airflow
│   └── requirements-airflow.txt
├── dashboard/                                # stretch goal, Section 13
│   └── app.py
├── docker-compose.yaml
├── .env.example
├── .gitignore
├── README.md
└── PROJECT_DESIGN.md                        # this document, kept in-repo for reference

```

10 Environment & Docker Compose Setup

Do not hand-write the Airflow core `docker-compose.yaml` from memory or from an old tutorial — the 3.x service layout differs from 2.x. Instead:

1. Fetch the current official compose file (this URL always resolves to the current stable release): `curl -Lf0 https://airflow.apache.org/docs/apache-airflow/stable/docker-compose.yaml`
2. Add a `docker/Dockerfile.airflow` that extends the base Airflow image and installs `docker/requirements-airflow.txt` (pandas, rapidfuzz, tenacity, pymongo, the three providers listed in Section 8, python-dotenv, pytest). Point the compose file's `image:/build:` at it.
3. Append two more services to the compose file — a warehouse Postgres, separate from

Airflow's own metadata Postgres, and Mongo:

```
warehouse-postgres:
  image: postgres:16
  environment:
    POSTGRES_USER: ${WAREHOUSE_POSTGRES_USER}
    POSTGRES_PASSWORD: ${WAREHOUSE_POSTGRES_PASSWORD}
    POSTGRES_DB: ${WAREHOUSE_POSTGRES_DB}
  ports: ["5433:5432"]      # 5433 host-side: 5432 is taken by Airflow's own metadata DB
  volumes: ["warehouse-db-volume:/var/lib/postgresql/data"]

mongo:
  image: mongo:7
  environment:
    MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER}
    MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}
  ports: ["27017:27017"]
  volumes: ["mongo-data:/data/db"]

mongo-express:          # dev-only admin UI, optional
  image: mongo-express
  ports: ["8081:8081"]
  depends_on: [mongo]

adminer:                 # dev-only admin UI for Postgres, optional
  image: adminer
  ports: ["8082:8080"]
```

4. Create .env from .env.example (below), mkdir -p dags logs plugins include data/raw, then docker compose up airflow-init followed by docker compose up -d.
5. Register secrets as Airflow Connections/Variables (via the UI, or airflow connections add / airflow variables set inside the scheduler container) — **never hardcode the WAQI token or DB passwords in DAG code.**
6. Apply the SQL in sql/ to warehouse-postgres (psql, Adminer, or a one-off schema_init DAG run) before triggering the pipeline DAGs.
7. In the Airflow UI: unpause and trigger dim_city_pipeline first (it must run at least once before facts can be resolved to cities), then unpause weather_aqi_pipeline.

.env.example

```
AIRFLOW_UID=50000
# Pin to the current stable tag – check hub.docker.com/r/apache/airflow before building
AIRFLOW_IMAGE_NAME=apache/airflow:3.x.x-python3.12
```

```
WAREHOUSE_POSTGRES_USER=dwh_user
WAREHOUSE_POSTGRES_PASSWORD=change_me
WAREHOUSE_POSTGRES_DB=weather_aqi_dwh
```

```
MONGO_USER=mongo_user
MONGO_PASSWORD=change_me
MONGO_DB=weather_aqi_raw
```

```
WAQI_API_TOKEN=get_your_own_token_from_aqicn.org_data-platform_token
OPEN_METEО_BASE_URL=https://api.open-meteo.com/v1/forecast
```

Commit .env.example, never commit .env. Add .env, logs/, and data/raw/*.csv (if it contains anything beyond the intentionally-committed sample) to .gitignore as appropriate — but note the messy CSV itself *should* be committed since it's a deliberate, documented test fixture, not a secret.

11 Testing Plan

- **Unit tests** (tests/test_city_matcher.py, test_clean_weather.py, test_clean_aqi.py, test_dq_checks.py) — pure functions, no live services, no network. Use responses or pytest-mock to stub HTTP calls in extract-function tests so the suite runs offline and deterministically. These are the tests to point an interviewer at.
- **DAG integrity test** (test_dag_integrity.py) — import both DAG files and assert zero import errors and the expected task count/structure. This is the automated equivalent of airflow dags list-import-errors and is exactly the kind of lightweight check worth running in CI.
- **Optional integration test** — a docker-compose-based test that runs one full DAG execution against a disposable schema and asserts row counts landed as expected. Nice to have, not required for v1.

12 Build Plan (recommended order, with acceptance criteria)

Build in this order. Each milestone should be independently verifiable before moving to the next — don't let the coding agent jump ahead.

Milestone 0 — Scaffold. Repo structure from Section 9, .env.example, Dockerfile, extended compose file. *Done when:* docker compose up -d brings up Airflow, warehouse-postgres, and mongo with no errors, and the Airflow UI loads at localhost:8080.

Milestone 1 — Warehouse schema. All SQL in sql/ applied to warehouse-postgres. *Done when:* all 8 tables exist with correct constraints (verify the two UNIQUE (city_id, reading_ts) constraints specifically).

Milestone 2 — Dimension pipeline. dim_city_pipeline DAG: read messy CSV → clean → resolve aliases → upsert dim_city/dim_city_alias. *Done when:* running it against the messy CSV from Section 4.3 produces the correct canonical city count with zero duplicate cities, and every documented messy pattern is demonstrably handled (write a test asserting this).

Milestone 3 — Weather extraction + raw load. extract_weather_batch + load_raw_weather tasks only. *Done when:* one manual DAG run populates raw_weather_readings in Mongo with real data for all active cities.

Milestone 4 — AQI extraction (dynamic mapping) + raw load. extract_aqi_city.expand(...) + load_raw_aqi. *Done when:* a manual run shows one mapped task instance per city in the Airflow UI's Grid view, and simulating a failure for one city (e.g. temporarily break its coordinates) doesn't fail the other mapped instances.

Milestone 5 — Transform + data quality gates. Both transform tasks, both DQ gates, quarantine writes. *Done when:* feeding it a batch that includes an out-of-range temperature and an unresolvable city name correctly produces rows in quarantine_unmatched_city_names / a DQ fail record, and doesn't load the bad rows into the fact tables.

Milestone 6 — Curated load + idempotency + run log. Upserts into both fact tables, pipeline_run_log write. *Done when:* running the DAG twice for the same logical_date results in the same row count in both fact tables (not doubled), and pipeline_run_log shows two entries.

Milestone 7 — Tests + README. All unit tests passing, DAG integrity test passing, README written per Section 14's talking points and Section 4's attribution requirements.

Milestone 8 (stretch, optional) — see Section 13.

13 Stretch Goals (post-v1, optional)

Only attempt these once Milestones 0–7 are solid and demonstrable:

- **Dashboard.** A small Streamlit app (`dashboard/app.py`) reading from warehouse-postgres — a live AQI map/table for the tracked cities plus a temperature trend chart. Makes the project demo-able in 30 seconds in an interview.
- **Secondary cloud load.** Add a task loading the same curated data into BigQuery (free tier) — lets you legitimately claim “one cloud platform, chosen deliberately” per the roadmap, without making it a hard dependency of the core pipeline.
- **Great Expectations.** Swap the hand-written `dq_checks.py` for a Great Expectations suite — mention in the README that you built the custom version first and understand exactly what it’s checking, which is arguably the stronger interview answer either way.
- **SCD Type 2 on `dim_city`.** Track historical changes to population/tier over time with `valid_from/valid_to` columns — most fresher projects skip this entirely.
- **Alerting.** `on_failure_callback` posting to Slack or sending email when a task exhausts retries.
- **CI.** A GitHub Actions workflow running `pytest`, a linter (`ruff`), and the DAG-integrity test on every push.

14 Resume & Interview Talking Points

Draft resume bullets (fill in real numbers once the pipeline has run for a while — do not fabricate metrics):

- “Built and orchestrated a multi-source ETL pipeline with Apache Airflow, ingesting live weather and air-quality data for 20 Indian cities every 3 hours; landed raw JSON in MongoDB and loaded a validated, idempotent star schema into PostgreSQL.”
- “Designed a 10-check automated data-quality layer (schema-drift, range, null, freshness, uniqueness checks) that quarantines bad records instead of silently loading them, plus a fuzzy-matching layer reconciling 6+ historical Indian city name variants against a canonical dimension table.”
- “Implemented exponential-backoff retries and graceful partial-failure handling via Airflow dynamic task mapping, so a single unresponsive API endpoint degrades rather than fails the pipeline; processed [X] readings over [N] days with zero duplicate records via idempotent upserts.”

Interview questions this project should let you answer fluently (mapped to what the roadmap notes interviewers actually probe for):

- “Walk me through what happens if this API is down when the DAG runs.” → Section 7.3.
- “How do you make sure rerunning this doesn’t duplicate data?” → Section 7.4.
- “How would you know if this pipeline silently started producing bad data?” → Section 6.4 / 7.5.
- “Why two databases instead of one?” → Section 3.
- “Show me a messy row and walk me through what your code does with it.” → have the messy CSV and `city_matcher.py` open and ready.
- “Write me a window function from memory” / “read this 200-line query” → practice with Section 15 below using your own warehouse data.

15 Example Analytical Queries (for SQL practice on your own data)

The roadmap notes SQL depth — window functions, CTEs — is explicitly screened for. These run directly against your own warehouse once it has a few days of data, so you can practice with data you understand end-to-end.

24-hour rolling average temperature per city (window function):

```
SELECT
  city_id,
  reading_ts,
  temperature_c,
  AVG(temperature_c) OVER (
    PARTITION BY city_id
    ORDER BY reading_ts
    RANGE BETWEEN INTERVAL '24 hours' PRECEDING AND CURRENT ROW
  ) AS rolling_24h_avg_temp
FROM fact_weather_reading
ORDER BY city_id, reading_ts;
```

Cities in “Unhealthy” AQI (>150) for 3+ consecutive readings — classic gaps-and-islands pattern:

```
WITH flagged AS (
  SELECT
    city_id, reading_ts, aqi_value,
    (aqi_value > 150) AS is_unhealthy,
    ROW_NUMBER() OVER (PARTITION BY city_id ORDER BY reading_ts)
      - ROW_NUMBER() OVER (
        PARTITION BY city_id, (aqi_value > 150)
        ORDER BY reading_ts
      ) AS island_id
  FROM fact_air_quality_reading
),
streaks AS (
  SELECT city_id, island_id, COUNT(*) AS streak_len,
    MIN(reading_ts) AS streak_start, MAX(reading_ts) AS streak_end
  FROM flagged
  WHERE is_unhealthy
  GROUP BY city_id, island_id
)
SELECT c.city_name, s.streak_len, s.streak_start, s.streak_end
FROM streaks s
JOIN dim_city c USING (city_id)
WHERE s.streak_len >= 3
ORDER BY s.streak_len DESC;
```

Rank cities by worst average AQI this week:

```
SELECT
  c.city_name,
  ROUND(AVG(f.aqi_value), 1) AS avg_aqi,
  RANK() OVER (ORDER BY AVG(f.aqi_value) DESC) AS worst_aqi_rank
FROM fact_air_quality_reading f
JOIN dim_city c USING (city_id)
WHERE f.reading_ts >= now() - INTERVAL '7 days'
GROUP BY c.city_name
ORDER BY worst_aqi_rank;
```

Practice explaining the query plan (EXPLAIN ANALYZE) for at least one of these — “query optimization” is explicitly named in the roadmap’s skill list.

16 Data Usage & Attribution

Include a short section in the README’s footer:

- **Weather data:** Open-Meteo (open-meteo.com) — free for non-commercial use.
- **Air quality data:** World Air Quality Index Project (waqi.info / aqicn.org), sourced from CPCB monitoring stations in India — attribution required by their terms of service; data not resold, not used in a paid product, not redistributed as an archived dataset. This project is a personal, non-commercial portfolio piece.
- **City metadata:** base figures from [cite your actual chosen public source]; messiness in the raw CSV is synthetic, added deliberately to exercise the cleaning pipeline (documented in Section 4.3).

End of specification.